

TeenPle 개발 표준 정의서



이 문서는 **TeenPle(SchoolHubYou)** 프로젝트의 백엔드 개발 및 배포 과정에서 **일관된 코드 작성, 협업 효율, 유지보수성**을 확보하기 위한 개발 표준을 정의한다.

- 본 표준은 **실제 서비스 배포를 전제로 한 기준**이다.
- 표준에 정의되지 않은 사항은 팀 논의를 통해 보완하고, 변경 사항은 문서에 반영한다.

1. 기본 원칙
2. 패키지 명명 규칙 (Package Naming Convention)
 - 2.1 기본 원칙
 - 2.2 최상위 패키지 구조
 - 2.3 도메인 패키지 규칙 (`domain/*`)
도메인 패키지 예시
 - 2.4 도메인 내부 하위 패키지 규칙
각 하위 패키지 역할
 - 2.5 글로벌 패키지 규칙 (`global/*`)
 - 2.6 패키지 설계 금지 사항
3. API 및 URL 명명 규칙
 - 3.1 URL 규칙
4. 예외 및 에러 처리 표준 (Exception & Error Handling)
 - 4.1 설계 원칙
 - 4.2 패키지 구조 표준
 - 4.2.1 Global 공통 (필수)
 - 4.2.2 Domain별 예외 (권장)
 - 4.3 에러 코드/메시지 규칙
 - 4.4 도메인 `ErrorStatus` 작성 규칙
 - 4.5 도메인 `Exception` 작성 규칙
 - 4.6 예외 발생 위치 및 사용 규칙 (서비스 레이어 표준)
 - 4.7 전역 예외 처리기(`ExceptionAdvice`) 표준
 - 4.8 공통 응답 포맷 표준 (성공/실패)
 - 4.9 도메인 `SuccessStatus` 운영 기준 (선택)
 - 4.10 금지 사항
5. 인증(JWT) 개발 표준
 - 5.1 JWT 적용 원칙
 - 5.2 JWT 미적용 API
6. Redis 사용 표준
 - 6.1 사용 목적
7. DB(RDS MySQL) 사용 표준
 - 7.1 저장 대상
 - 7.2 원칙
8. S3 파일 처리 표준
 - 8.1 Presigned URL 원칙
 - 8.2 업로드 대상
9. 주석 및 문서화 규칙 (Comment & Documentation)
 - 9.1 클래스 주석 규칙
클래스 주석 작성 예시
 - 9.2 메서드 주석 규칙

메서드 주석 작성 기준	
메서드 주석 예시	
9.3 인라인 주석 규칙	
10. Git 협업 규칙 (Collaboration Rules)	
10.1 브랜치 전략	
10.2 작업 브랜치 규칙	
브랜치 네이밍 규칙	
10.3 커밋 컨벤션	
Commit Type	
10.4 커밋 메시지 작성 규칙	
커밋 메시지 예시	
10.5 Pull Request 규칙	
11. 환경별 설정 관리 (Environment Configuration)	
11.1 환경 분리 원칙	
11.2 환경 구분	
11.3 설정 파일 관리	
11.4 민감 정보 관리	
관리 방법	
11.5 Docker 환경 설정	
12. 로그 관리 (Logging Policy)	
12.1 로그 작성 목적	
12.2 로그 레벨 기준	
12.3 환경별 로그 사용	
12.4 로그 보안	
13. 보안 기본 원칙 (Security Policy)	
13.1 통신 보안	
13.2 인증 및 인가	
13.3 토큰 관리	
13.4 파일 업로드 보안	
14. API 문서화 (Swagger)	
14.1 Swagger 사용 목적	
14.2 문서화 대상	
14.3 Swagger 작성 규칙	
14.4 환경별 Swagger 노출 정책	
14.5 문서 최신화 원칙	
15. 문서 개정 이력	

1. 기본 원칙

1. 본 문서에 정의된 개발 표준은 **모든 팀원이 준수**한다.
 2. 표준에 정의되지 않은 사항은 **팀 논의 후 문서에 반영**한다.
 3. 보안·안정성과 직결되는 항목(JWT, DB, 인증)은 **임의 변경을 금지**한다.
 4. 표준을 준수하지 못하는 경우, 반드시 **PR 또는 이슈로 사유를 공유**한다.
-

2. 패키지 명명 규칙 (Package Naming Convention)

2.1 기본 원칙

1. 모든 패키지명은 **소문자**로 작성한다.

2. 패키지명은 **단수형(singular)** 을 기본으로 한다.
3. 패키지 구조는 **역할 중심이 아닌 도메인 중심 설계**를 따른다.
4. 하나의 패키지는 **하나의 책임(도메인)** 을 가진다.
5. 패키지명은 의미가 명확해야 하며, 불필요한 축약어 사용을 지양한다.

2.2 최상위 패키지 구조

백엔드 패키지는 다음 두 영역으로 구분한다.

```
backend
├─ domain // 비즈니스 도메인 영역
└─ global // 전역 공통 영역
```

- **domain**
 - 서비스의 핵심 비즈니스 로직을 담당하는 도메인 패키지
- **global**
 - 전 도메인에서 공통으로 사용하는 설정, 공통 객체, 예외 처리 등을 포함

2.3 도메인 패키지 규칙 (**domain/***)

1. **domain** 하위에는 **기능(업무) 단위 도메인 패키지**를 생성한다.
2. 도메인 패키지명은 **서비스에서 사용하는 실제 용어**를 그대로 사용한다.
3. 서로 다른 도메인은 **직접 의존하지 않도록 설계**한다.
4. 도메인 간 협력이 필요한 경우, **서비스 레이어를 통해 간접적으로 처리**한다.

도메인 패키지 예시

```
domain
├─ auth
├─ board
├─ chatmessage
├─ chatroom
├─ chatroomuser
├─ comment
├─ media
├─ notification
├─ penalty
├─ post
├─ reaction
├─ region
├─ report
├─ school
├─ user
├─ usersetting
└─ verification
```

2.4 도메인 내부 하위 패키지 규칙

각 도메인 패키지는 다음과 같은 고정된 하위 구조를 따른다.

```
domain/{domain-name}
├─ controller
├─ service
├─ repository
├─ entity
├─ dto
├─ enums
└─ exception
```

각 하위 패키지 역할

- **controller** : 외부 요청을 처리하는 API 계층
- **service** : 비즈니스 로직 처리 계층
- **repository** : 데이터 접근 계층(JPA Repository)
- **entity** : JPA Entity 클래스
- **dto** : 요청/응답 데이터 전달 객체
- **enums** : 도메인에서 사용하는 열거형(Enum)
- **exception** : 도메인 전용 예외 클래스

2.5 글로벌 패키지 규칙 (**global/***)

- **global** 패키지는 모든 도메인에서 공통으로 사용하는 요소만 포함한다.
- 특정 도메인에 종속되는 로직은 **global**에 두지 않는다.

예시:

```
global
├─ apiPayload // 공통 API 응답 포맷
├─ code      // 공통 코드, 에러 코드
├─ config    // Security, JPA, Redis 설정
├─ exception // 전역 예외 처리
└─ util      // 공통 유틸리티
```

2.6 패키지 설계 금지 사항

- 도메인 패키지 하위에 다른 도메인을 포함 ❌
- **common**, **utils** 와 같은 포괄적 의미의 도메인 생성 ❌
- 하나의 패키지에 여러 도메인 책임 혼합 ❌

3. API 및 URL 명명 규칙

3.1 URL 규칙

- 소문자 사용
- `kebab-case` 사용
- URL에 동사 사용 ❌

예시:

```
POST /api/auth/signup
POST /api/auth/login
GET /api/boards
POST /api/schools/verify
```

```
POST /api/auth/signup
POST /api/auth/login
GET /api/boards
POST /api/schools/verify
```

4. 예외 및 에러 처리 표준 (Exception & Error Handling)

4.1 설계 원칙

1. 예외 처리는 **Global 공통 규격 + Domain 확장 규격**으로 구성한다.
2. 모든 API 에러 응답은 **단일 포맷(ApiResponse)** 을 사용한다.
3. 에러 코드는 "도메인별 Prefix + 숫자" 형태로 관리하며, **중복을 금지**한다.
4. 예외 발생 지점은 서비스/리포지토리에서 **즉시 throw**하고, Controller에서 try-catch로 처리하지 않는다.
5. 실제 응답 변환(HTTP Status + Body)은 **전역 예외 처리기(ExceptionAdvice)** 가 담당한다.

4.2 패키지 구조 표준

4.2.1 Global 공통 (필수)

- 공통 응답/코드/에러 DTO는 `global.apiPayload` 하위에서 관리한다.

예시:

```
global
├── apiPayload
│   ├── ApiResponse
│   ├── code
│   │   ├── BaseCode
│   │   ├── BaseErrorCode
│   │   ├── ReasonDto
│   │   ├── ErrorReasonDto
│   │   └── status
│   │       ├── SuccessStatus
│   │       └── ErrorStatus
│   └── exception
```

```
└─ ExceptionAdvice
└─ GeneralException
```

4.2.2 Domain별 예외 (권장)

- 도메인별로 "상태(enum) + 예외 클래스"를 둔다.

예시:

```
domain/post/exception
└─ PostException
└─ status
└─ PostErrorStatus
└─ PostSuccessStatus // 선택
```

4.3 에러 코드/메시지 규칙

1. 코드 형식: {DOMAIN}{4자리}

- 예: POST4000, USER4000

2. 도메인 Prefix는 도메인 패키지명과 일치시킨다. (post → POST)

3. 메시지는 사용자에게 전달 가능한 수준으로 작성하고, 내부 시스템 정보/스택트레이스는 포함하지 않는다.

4. HttpStatus는 실제 의미와 일치하도록 설정한다.

- 리소스 없음 → 404 NOT_FOUND
- 권한 문제 → 403 FORBIDDEN (권장) 또는 정책에 따라 400 (팀 기준 통일)

4.4 도메인 ErrorStatus 작성 규칙

- 도메인별 에러 상태는 enum XxxErrorStatus implements BaseErrorCode 형태로 작성한다.
- httpStatus, code, message 3요소를 반드시 포함한다.
- getReason() / getReasonHttpStatus()는 공통 DTO(ErrorReasonDto) 빌더 규격을 따른다.
- 현재 PostErrorStatus처럼 작성하는 방식을 표준으로 한다.

4.5 도메인 Exception 작성 규칙

- 도메인 예외는 XxxException extends GeneralException (또는 팀에서 사용하는 Base Exception) 형태로 작성한다.
- 생성자는 XxxErrorStatus (또는 BaseErrorCode)를 받아서 상위로 전달한다.
- 예외 클래스명은 도메인 Prefix를 포함한다.
 - 예: UserException, PostException, SchoolException

4.6 예외 발생 위치 및 사용 규칙 (서비스 레이어 표준)

1. 조회 실패는 orElseThrow()로 즉시 예외를 발생시킨다.
2. Controller는 예외를 처리하지 않고 그대로 전파한다.
3. 예외는 반드시 도메인 ErrorStatus를 사용한다. (문자열 하드코딩 금지)

예시(표준 사용 형태):

```
userRepository.findByIdWithSchoolAndRegion(req.getUserId())
    .orElseThrow(() -> new UserException(UserErrorStatus.USER_NOT_FOUND));
```

4.7 전역 예외 처리기(ExceptionAdvice) 표준

1. 전역 예외 처리기는 다음을 책임진다.

- 도메인 예외(`GeneralException` 계열)를 `ApiResponse` 형태로 변환
- `HttpStatus` 세팅
- 공통 에러 포맷 유지

2. 처리 대상(최소):

- 도메인 커스텀 예외(`GeneralException` 계열)
- Validation 예외 (`MethodArgumentNotValidException` 등)
- 예상치 못한 예외 (`Exception`) -> 공통 `INTERNAL_ERROR` 로 매핑

4.8 공통 응답 포맷 표준 (성공/실패)

- 모든 응답은 `ApiResponse` 규격을 따른다.
- 실패 응답에는 최소 아래 항목을 포함한다.
 - `isSuccess = false`
 - `code`
 - `message`
 - (선택) `httpStatus`

| BaseErrorCode -> ErrorReasonDto -> ApiResponse로 이어지는 흐름을 표준으로 한다.

4.9 도메인 SuccessStatus 운영 기준 (선택)

- 성공 코드까지 엄격히 관리하는 경우에만 도메인별 `SuccessStatus` 를 둔다.
- 도입 여부는 팀 합의에 따른다. (현재 구조처럼 global `SuccessStatus` 만 사용해도 무방)

4.10 금지 사항

- Controller에서 try-catch로 에러 응답을 직접 만드는 행위 ❌
- 예외 메시지를 문자열로 하드코딩하여 throw 하는 행위 ❌
- 도메인과 무관한 예외 코드를 global에 무분별하게 추가 ❌

5. 인증(JWT) 개발 표준

5.1 JWT 적용 원칙

- 로그인 성공 후 JWT 발급
- Access Token은 API 인증용
- Refresh Token은 Redis에 저장

5.2 JWT 미적용 API

다음 API는 JWT 인증을 요구하지 않는다.

- 회원가입
 - 로그인
 - Presigned URL 발급 API
-

6. Redis 사용 표준

Redis는 영속 데이터 저장소로 사용하지 않는다.

6.1 사용 목적

- Refresh Token 저장
 - 토큰 블랙리스트 관리
 - 임시 인증 데이터
 - 채팅 Pub/Sub (사용 시)
-

7. DB(RDS MySQL) 사용 표준

※ 이 섹션은 문서상 “수정 필요”로 표시되어 있어, 추후 팀 합의로 보완한다.

7.1 저장 대상

- 사용자 정보
- 학교 인증 상태
- 게시글 / 댓글
- 채팅 로그(필요 시)

7.2 원칙

- 비즈니스 핵심 데이터만 저장
 - Redis와 역할 중복 금지
-

8. S3 파일 처리 표준

8.1 Presigned URL 원칙

- 이미지 파일은 서버를 거치지 않는다.
- 클라이언트 → S3 직접 업로드

8.2 업로드 대상

- 학생증 이미지
 - 프로필 이미지
-

9. 주석 및 문서화 규칙 (Comment & Documentation)

9.1 클래스 주석 규칙

1. 모든 **Controller, Service, Entity, Exception** 클래스에는 클래스 상단 주석을 작성한다.
2. 주석은 **JavaDoc 형식**(`/**` / `*/`)* 을 사용한다.
3. 클래스 주석에는 다음 항목을 포함한다.
 - 클래스의 역할 및 책임
 - 주요 기능 요약
 - 필요 시 연관 도메인 또는 사용 위치

클래스 주석 작성 예시

```
/**
 * 게시글(Post) 도메인의 비즈니스 로직을 처리하는 서비스 클래스
 *
 * 게시글 생성, 조회, 수정, 삭제 기능을 담당한다.
 */
@Service
public class PostService {
    ...
}
```

- 단순 DTO, Enum 등 의미가 명확한 클래스는 생략할 수 있다.
- 도메인 예외(`*Exception` , `*ErrorStatus`)는 주석 작성을 권장한다.

9.2 메서드 주석 규칙

1. 모든 메서드에 주석을 강제하지는 않는다.
2. 다음 경우에는 **반드시 메서드 주석을 작성한다**.
 - 비즈니스 로직이 복잡한 경우
 - 조건 분기/권한 체크가 많은 경우
 - 외부 도메인과 협력하는 로직
3. 주석은 **메서드의 의도와 핵심 흐름 중심**으로 작성한다.

메서드 주석 작성 기준

- 메서드의 목적
- 주요 파라미터 의미
- 반환값 의미
- 발생 가능한 예외(중요한 경우)

메서드 주석 예시

```

/**
 * 게시글을 삭제한다.
 *
 * 작성자 본인만 삭제 가능하며,
 * 이미 삭제된 게시글은 예외를 발생시킨다.
 *
 * @param postId 삭제할 게시글 ID
 * @param userId 요청 사용자 ID
 * @throws PostException 게시글이 존재하지 않거나 권한이 없는 경우
 */
public void deletePost(Long postId, Long userId) {
    ...
}

```

- 단순 Getter/Setter, Repository 메서드는 주석을 생략할 수 있다.

9.3 인라인 주석 규칙

1. 복잡한 조건문, 분기 로직 위에 한 줄 주석(//)으로 의도를 설명한다.
2. "무엇을 하는 코드인지"가 아니라 **"왜 이런 처리를 하는지"** 를 설명한다.

예시:

```

// 학생 인증이 완료되지 않은 사용자는 게시글 작성 불가
if (!user.isVerified()) {
    throw new UserException(UserErrorStatus.USER_NOT_VERIFIED);
}

```

10. Git 협업 규칙 (Collaboration Rules)

10.1 브랜치 전략

브랜치	설명
main	실제 운영(프로덕션) 배포 브랜치
develop	기능 통합 및 개발 기준 브랜치
demo	시연 및 테스트 서버 배포용 브랜치

10.2 작업 브랜치 규칙

- 모든 작업은 **이슈 단위 브랜치**로 진행한다.
- 브랜치는 반드시 **develop** 기준으로 생성한다.

브랜치 네이밍 규칙

```

feat/{이슈번호}-{간단설명}
fix/{이슈번호}-{간단설명}

```

```
docs/{이슈번호}-{간단설명}
chore/{이슈번호}-{간단설명}
```

예시:

```
feat/12-post-create
fix/21-login-token-error
docs/30-update-readme
```

10.3 커밋 컨벤션

커밋 메시지는 **README**에 정의된 컨벤션을 따른다.

Commit Type

- **Feat** : 새로운 기능 추가
- **Fix** : 버그 수정
- **Docs** : 문서 수정
- **Refactor** : 리팩토링
- **Chore** : 빌드/설정/기타 작업
- **Test** : 테스트 코드
- **Style** : 포맷 수정 (로직 변경 없음)

10.4 커밋 메시지 작성 규칙

1. 제목은 **50자 이내**, 마침표 사용 ❌
2. 변경 내용을 **간결한 명령형**으로 작성한다.
3. 필요 시 Body에 “무엇을 / 왜” 변경했는지 작성한다.
4. PR 생성 시 이슈 번호를 연결한다.

커밋 메시지 예시

Feat: Add post delete feature

게시글 삭제 기능 추가
작성자 권한 검증 로직 포함

Resolves: #42

10.5 Pull Request 규칙

1. 모든 PR은 **develop** 브랜치를 대상으로 한다.
2. PR 본문에는 다음을 포함한다.
 - 작업 내용 요약

- 관련 이슈 번호

3. 코드 리뷰 후 머지를 원칙으로 한다.

11. 환경별 설정 관리 (Environment Configuration)

11.1 환경 분리 원칙

실행 환경에 따라 설정을 분리하여 관리한다. 이는 **보안 강화**, **배포 안정성**, **운영 편의성 확보**를 목적으로 한다.

11.2 환경 구분

다음 실행 환경을 정의한다.

환경	설명
local	개발자 개인 로컬 환경
dev	팀 개발 및 테스트 환경
prod	실제 서비스 운영 환경

11.3 설정 파일 관리

- Spring Profile 기반 설정을 사용한다.
- 환경별 설정 파일은 다음 규칙을 따른다.

```
application.yml
application-local.yml
application-dev.yml
application-prod.yml
```

- 실행 환경은 `spring.profiles.active` 값으로 지정한다.

11.4 민감 정보 관리

다음 정보는 **코드에 직접 작성하지 않는다**.

- DB URL / 계정 / 비밀번호
- JWT Secret Key
- AWS Access Key / Secret Key
- Redis 접속 정보

관리 방법

- 환경 변수 또는 `.env` 파일로 주입한다.
- Git 저장소에 민감 정보를 포함하지 않는다.
- `.env`, `application-prod.yml` 등은 `.gitignore`에 등록한다.

11.5 Docker 환경 설정

- Docker 컨테이너 실행 시 환경 변수를 주입한다.
- 환경별 설정 차이는 컨테이너 이미지가 아닌 **실행 옵션으로 관리**한다.

12. 로그 관리 (Logging Policy)

12.1 로그 작성 목적

로그는 다음을 위해 작성한다.

- 서비스 장애 원인 추적
- 비정상 사용자 행위 분석
- 운영 상태 파악

12.2 로그 레벨 기준

레벨	사용 기준
ERROR	시스템 장애, 예외 발생
WARN	비정상 가능성이 있는 흐름
INFO	주요 비즈니스 이벤트
DEBUG	개발 단계 디버깅용

12.3 환경별 로그 사용

- **local**, **dev** 환경: **INFO**, **DEBUG** 허용
- **prod** 환경: **DEBUG** 로그 사용 금지

12.4 로그 보안

다음 정보는 **로그에 출력하지 않는다**.

- 비밀번호
- JWT 토큰 원문
- 인증 코드
- 개인정보(전화번호, 학생증 정보 등)

👉 필요 시 마스킹 처리한다.

13. 보안 기본 원칙 (Security Policy)

13.1 통신 보안

- 모든 API 통신은 **HTTPS**를 사용한다.
- HTTP 접근은 허용하지 않는다.

13.2 인증 및 인가

- 인증이 필요한 API는 JWT 인증 필터를 거친다.
 - 회원가입/로그인/Presigned URL 발급 API는 JWT 인증을 적용하지 않는다.
 - 사용자 권한(Role)에 따라 접근 가능한 API를 구분한다.
-

13.3 토큰 관리

- Access Token은 짧은 만료 시간을 가진다.
 - Refresh Token은 Redis에 저장하여 관리한다.
 - 로그아웃 시 Refresh Token을 즉시 무효화한다.
-

13.4 파일 업로드 보안

- 파일 업로드는 Presigned URL 방식만 허용한다.
 - 서버가 파일 데이터를 직접 수신하지 않는다.
 - Presigned URL에는 만료 시간을 설정한다.
-

14. API 문서화 (Swagger)

14.1 Swagger 사용 목적

- API 명세 공유
 - 프론트엔드 협업
 - 테스트 및 검증 편의성 확보
-

14.2 문서화 대상

다음 API는 Swagger에 문서화한다.

- 외부에 공개되는 모든 REST API
 - 인증이 필요한 API
 - 주요 Request / Response 구조
-

14.3 Swagger 작성 규칙

- 모든 Controller에 API 설명을 작성한다.
 - Request Body, Path Variable, Query Parameter를 명확히 명시한다.
 - 성공/실패 응답 예시를 포함한다.
 - 인증이 필요한 API는 JWT 인증 필요 여부를 명시한다.
-

14.4 환경별 Swagger 노출 정책

- `local`, `dev` 환경: Swagger UI 활성화
- `prod` 환경: Swagger UI 비활성화 (또는 접근 제한)

14.5 문서 최신화 원칙

- API 변경 시 Swagger 문서를 함께 수정한다.
- 코드와 문서 불일치 상태를 허용하지 않는다.

본 개발 표준 정의서는 TeenPle 프로젝트의 실제 서비스 배포를 전제로 하여 개발, 보안, 운영, 협업 전반에 대한 기준을 정의한다.

모든 팀원은 본 표준을 준수하며, 변경 사항은 팀 논의를 통해 지속적으로 반영한다.

15. 문서 개정 이력

버전	날짜	변경 내용	작성자
1.0	2026-01-13	초안 작성	홍왕기

본 개발 표준은 TeenPle 프로젝트의 품질과 안정성을 보장하는 핵심 기준이다.

표준을 지키는 것은 개인의 편의가 아닌, 팀 전체의 생산성과 서비스 신뢰성을 위한 것이다.